

Proyek 3 Proyek Pengembangan Perangkat Lunak Berbasis Web

Modul 2

Vanilla JavaScript dan Interaktivitas Web



Daftar Isi

Daftar Isi	2
1. Modul Overview	3
1.1 Tujuan Pembelajaran	3
1.2 Materi Esensial	3
1.3 Target Milestones	3
1.4 Alur Belajar yang Digunakan	4
2. Materi untuk Praktikum	4
2.1 Peran JavaScript dan Lingkungan Kerja	4
2.2 Variabel Tipe Data Operator dan Konversi	5
2.3 Percabangan dan Perulangan	6
2.4 Function dan Ruang Lingkup	7
2.5 Array dan Object	8
2.6 DOM dan Manipulasi Halaman	9
2.7 Event dan Interaksi Pengguna	9
2.8 Validasi Form	16
2.9 Asynchronous JavaScript	17
2.10 Debugging dan Troubleshooting	25
2.11 Integrasi dan Checklist Kualitas	30
3. Tugas Praktikum	31
4. Mini Project : Landing Page Interaktif	40
4.1 Brief Produk	40
4.2 Fitur Minimal	40
4.3 Batas Teknis	40
4.4 Tahapan Pengerjaan	40
4.5 Definition of Done	41
4.6 Acceptance Criteria	41
4.7 Batas Penggunaan AI	41
5. Penilaian dan Refleksi	42
5.1 Kuesioner Self Assessment	42
5.2 Peer Assessment	42
5.3 Lesson Learnt	42
6. Rubrik Penilaian Dosen	43
7. Referensi	43

1. Modul Overview

Modul ini digunakan pada mata kuliah Proyek 3 Proyek Pengembangan Perangkat Lunak Berbasis Web di Program Studi D3 Teknik Informatika. Setelah membangun landing page statis pada Modul 1, mahasiswa menambahkan perilaku pada halaman menggunakan JavaScript. Fokus modul adalah fondasi bahasa, DOM, event, validasi form, asynchronous JavaScript menggunakan Promise, async/await, dan fetch, serta debugging. Module dan framework dibahas pada tahap berikutnya.

1.1 Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Menjelaskan peran JavaScript dalam hubungan HTML, CSS, DOM, dan browser.
2. Menggunakan const, let, tipe data, operator, percabangan, perulangan, dan function.
3. Mengolah kumpulan data sederhana menggunakan array dan object.
4. Memilih elemen, mengubah konten atau class, dan membuat elemen melalui DOM.
5. Menangani event pengguna serta membuat validasi form yang memberikan umpan balik jelas.
6. Menjelaskan alur asynchronous JavaScript serta menggunakan Promise, async/await, dan fetch untuk membaca data JSON.
7. Membaca pesan error, menelusuri nilai, dan memperbaiki masalah menggunakan Console dan Network pada DevTools.
8. Mengembangkan landing page interaktif yang dapat diuji dan dijelaskan tanpa bergantung pada AI.

1.2 Materi Esensial

1. JavaScript menjalankan perilaku; HTML tetap menyimpan struktur dan CSS tetap mengatur presentasi.
2. Nilai memiliki tipe. Operator, perbandingan ketat, dan konversi tipe harus digunakan secara sadar.
3. Percabangan memilih tindakan; perulangan menangani pekerjaan yang berpola.
4. Function memisahkan tanggung jawab dan mengembalikan hasil melalui return.
5. Array menyimpan daftar; object menyimpan atribut suatu entitas.
6. DOM adalah representasi dokumen yang dapat dibaca dan diubah oleh JavaScript.
7. Event menghubungkan tindakan pengguna dengan function.
8. Proses asynchronous menggunakan Promise; async/await mempermudah alur, sedangkan fetch membaca sumber data dan harus menangani kegagalan.
9. Debugging dimulai dari bukti: pesan Console, status Network, nilai aktual, dan langkah reproduksi.

1.3 Target Milestones

Milestone	Indikator Keberhasilan
Script Ready	File app.js terhubung menggunakan defer dan tidak menghasilkan error awal.
Logic Builder	Program memakai tipe data, kondisi, perulangan, dan function dengan benar.
Data Organizer	Array dan object dapat dibaca, diubah, serta ditampilkan.
DOM Controller	Konten dan class berubah melalui selector serta event listener.
Form Validator	Input tidak valid ditolak dengan pesan yang spesifik dan mudah dipahami.
Async Data Loader	Promise, async/await, fetch, serta state loading, success, empty, dan error dapat diterapkan dan diuji.
Mini Project	Landing page interaktif selesai, dapat diuji, dan dapat dijelaskan.

Aturan AI: Kerjakan prediksi dan percobaan awal sendiri. AI hanya boleh digunakan untuk meminta penjelasan konsep, membaca pesan error, atau memperoleh hint bertahap setelah ada bukti percobaan. AI tidak boleh menghasilkan seluruh task atau mini project. Mahasiswa harus mampu menjelaskan dan mengubah kode saat implementasi.

1.4 Alur Belajar yang Digunakan

Tahap	Aktivitas Mahasiswa	Bukti
Predict	Menuliskan perkiraan output atau perilaku.	Prediksi singkat
Run	Menjalankan satu perubahan kecil.	Kode dan output
Observe	Membandingkan prediksi dengan hasil.	Catatan pengamatan
Explain	Menjelaskan penyebab hasil.	Jawaban dengan kata sendiri
Improve	Memperbaiki kode dan menguji ulang.	Versi akhir

2. Materi untuk Praktikum

2.1 Peran JavaScript dan Lingkungan Kerja

JavaScript berjalan di browser dan dapat membaca dokumen yang telah dibentuk menjadi DOM. Ketika JavaScript mengubah DOM atau class CSS, browser memperbarui tampilan. Untuk latihan ini, simpan perilaku dalam file eksternal agar struktur, tampilan, dan perilaku tetap terpisah.

A. Hubungan HTML CSS dan JavaScript

Lapisan	Peran	Contoh
HTML	Struktur dan makna konten	button, form, section
CSS	Tampilan dan layout	warna, spacing, responsive
JavaScript	Perilaku dan logika	klik, validasi, perubahan konten
DOM	Representasi halaman di memori browser	document.querySelector

B. Menghubungkan File JavaScript

```
<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Latihan JavaScript</title>
  <script src="js/app.js" defer></script>
</head>
<body>
  <h1>JavaScript siap</h1>
</body>
</html>
```

Atribut defer meminta browser mengunduh script tanpa menghentikan pembacaan HTML, lalu menjalankannya setelah dokumen selesai diparse. Dengan demikian, elemen HTML sudah tersedia ketika selector dijalankan.

C. Console sebagai Alat Observasi

```
console.log('Script terhubung');
console.log({ halaman: document.title, status: 'siap' });
```

Buka DevTools melalui F12 atau menu Inspect, kemudian pilih tab Console. Console digunakan untuk melihat pesan, nilai variabel, dan error. Jangan menghapus error sebelum memahami sumbernya.

Eksperimen 1. Console dan Tipe Data

Tujuan eksperimen: membedakan tipe data, memprediksi hasil operator, dan mengenali konversi implisit.

```
const nama = 'Rani';
const nilaiTeks = '80';
const nilaiAngka = 80;
const aktif = true;

console.log(typeof nama);
console.log(typeof nilaiTeks);
console.log(typeof nilaiAngka);
console.log(typeof aktif);
console.log(nilaiTeks + 5);
console.log(Number(nilaiTeks) + 5);
console.log(nilaiTeks == nilaiAngka);
console.log(nilaiTeks === nilaiAngka);
```

1. Prediksi setiap output sebelum membuka Console.
2. Jalankan kode dan catat hasil aktual.
3. Ubah nilaiTeks menjadi string yang bukan angka, lalu ulangi Number(nilaiTeks).
4. Jelaskan mengapa operator + dapat menghasilkan hasil berbeda.
5. Simpulkan alasan menggunakan perbandingan ketat === sebagai pilihan awal.

Pertanyaan pemahaman:

- 1) Apa bukti bahwa nilai 80 dan teks "80" berbeda?
- 2) Apa yang dihasilkan Number ketika teks tidak dapat dikonversi?
- 3) Mengapa prediksi harus ditulis sebelum kode dijalankan?

2.2 Variabel Tipe Data Operator dan Konversi

A. const dan let

Gunakan const ketika referensi variabel tidak perlu diarahkan ke nilai lain. Gunakan let ketika nilainya memang berubah. Hindari var pada latihan dasar agar aturan scope lebih mudah dipahami.

```
const namaKelas = 'D3 TI';
let jumlahPeserta = 24;

jumlahPeserta = jumlahPeserta + 1;
console.log(`${namaKelas}: ${jumlahPeserta} peserta`);
```

B. Tipe Data Dasar

Tipe	Contoh	Catatan
string	'Bandung'	Teks diapit tanda kutip.
number	75 atau 3.14	Angka bulat dan desimal menggunakan tipe number.
boolean	true atau false	Mewakili kondisi logis.
undefined	let hasil;	Variabel belum memiliki nilai.

Tipe	Contoh	Catatan
null	const data = null;	Ketiadaan nilai yang diberikan secara sengaja.

C. Operator dan Perbandingan

```
const harga = 25000;
const jumlah = 3;
const subtotal = harga * jumlah;
const mendapatDiskon = subtotal >= 50000;
const stokCukup = jumlah <= 5;

console.log({ subtotal, mendapatDiskon, stokCukup });
console.log(mendapatDiskon && stokCukup);
```

Operator && bernilai benar ketika kedua kondisi benar. Operator || bernilai benar ketika setidaknya satu kondisi benar. Operator ! membalik nilai boolean. Gunakan tanda kurung jika susunan kondisi mulai sulit dibaca.

D. Konversi Input

Nilai dari elemen input dibaca sebagai string, termasuk ketika type bernilai number. Konversikan secara eksplisit dan periksa hasil sebelum melakukan perhitungan.

```
const jumlahInput = document.querySelector('#jumlah');
const jumlah = Number(jumlahInput.value);

if (Number.isNaN(jumlah)) {
  console.log('Jumlah bukan angka yang valid');
}
```

Pertanyaan pemahaman:

- 1) Kapan let lebih tepat daripada const?
- 2) Mengapa nilai input perlu dikonversi?
- 3) Apa perbedaan null dan undefined dalam contoh praktis?

2.3 Percabangan dan Perulangan

A. Membuat Keputusan dengan if

```
function hitungDiskon(subtotal) {
  if (subtotal >= 200000) {
    return 0.2;
  }

  if (subtotal >= 100000) {
    return 0.1;
  }

  return 0;
}
```

Urutan kondisi penting. Pemeriksaan nilai terbesar ditempatkan lebih dahulu agar aturan diskon tidak tertutup oleh kondisi yang lebih umum.

B. Perulangan Daftar

```
const topik = ['Variabel', 'Function', 'DOM'];

for (const item of topik) {
  console.log(`Belajar: ${item}`);
}
```

Gunakan for...of ketika kebutuhan utamanya membaca setiap item. Gunakan for klasik ketika indeks atau perubahan langkah diperlukan. Pastikan kondisi berhenti dapat tercapai agar tidak terjadi infinite loop.

Pertanyaan pemahaman:

- 1) Mengapa kondisi diskon terbesar diperiksa lebih dulu?
- 2) Kapan perulangan lebih baik daripada menulis kode berulang?
- 3) Apa penyebab umum infinite loop?

2.4 Function dan Ruang Lingkup

Function adalah blok kode yang mempunyai satu tanggung jawab. Parameter menerima data, sedangkan return mengirimkan hasil kepada pemanggil. Nama function sebaiknya menggunakan kata kerja yang menjelaskan tindakannya.

A. Parameter dan Return

```
function hitungTotal(harga, jumlah, diskon) {
  const subtotal = harga * jumlah;
  return subtotal - subtotal * diskon;
}

const total = hitungTotal(25000, 3, 0.1);
console.log(total);
```

B. Memisahkan Logika dan Tampilan

```
function tentukanStatus(nilai) {
  return nilai >= 60 ? 'Lulus' : 'Belum lulus';
}

function tampilkanStatus(elemen, status) {
  elemen.textContent = status;
}
```

Function tentukanStatus hanya mengolah data sehingga mudah diuji melalui Console. Function tampilkanStatus hanya memperbarui tampilan. Pemisahan ini mempermudah pencarian masalah.

C. Scope

```
const namaProgram = 'Proyek 3';

function buatPesan(nama) {
  const pesan = `Halo, ${nama}`;
  return `${pesan} - ${namaProgram}`;
}

console.log(buatPesan('Dina'));
// console.log(pesan); // ReferenceError
```

Variabel pesan hanya tersedia di dalam function. Batasi variabel pada scope terkecil yang memerlukannya agar perubahan tidak menyebar ke bagian lain.

Pertanyaan pemahaman:

- 1) Apa perbedaan parameter, argument, dan return?
- 2) Mengapa function logika dipisahkan dari function tampilan?
- 3) Mengapa variabel pesan tidak dapat diakses dari luar function?

2.5 Array dan Object

A. Array sebagai Daftar

```
const keterampilan = ['HTML', 'CSS'];
keterampilan.push('JavaScript');
for (const keterampilanItem of keterampilan) {
  console.log(keterampilanItem);
}
```

B. Object sebagai Entitas

```
const mahasiswa = {
  nama: 'Alya',
  programStudi: 'D3 Teknik Informatika',
  aktif: true,
  keterampilan: ['HTML', 'CSS', 'JavaScript']
};
console.log(mahasiswa.nama);
console.log(mahasiswa.keterampilan[0]);
```

C. Array of Objects

```
const fitur = [
  { judul: 'Cepat', deskripsi: 'Interaksi tanpa memuat ulang.' },
  { judul: 'Jelas', deskripsi: 'Umpan balik mudah dipahami.' },
  { judul: 'Teruji', deskripsi: 'Perilaku memiliki skenario uji.' }
];
for (const item of fitur) {
  console.log(`${item.judul}: ${item.deskripsi}`);
}
```

Gunakan array of objects ketika halaman perlu menampilkan beberapa entitas dengan struktur yang sama. Data tetap terpisah dari aturan untuk menampilkannya.

Pertanyaan pemahaman:

- 1) Mengapa variable fitur lebih tepat menjadi array of objects?
- 2) Apa akibat mengakses indeks yang tidak tersedia?
- 3) Kapan push digunakan?

Independent Challenge 1. Kalkulator Harga

Buat program Console yang menerima data harga, jumlah, dan status anggota melalui variabel. Program menghitung subtotal, menentukan diskon, dan menampilkan ringkasan pembayaran.

- Subtotal minimal Rp200.000 memperoleh diskon 20%.
- Subtotal minimal Rp100.000 memperoleh diskon 10%.
- Anggota memperoleh tambahan diskon 5%, tetapi total diskon maksimal 25%.
- Harga dan jumlah harus lebih besar dari nol.
- Perhitungan harus dipisahkan ke dalam function dan menghasilkan return value.

Bukti keberhasilan: tampilkan sedikitnya lima kasus uji, termasuk nilai batas Rp100.000 dan Rp200.000 serta satu input tidak valid.

2.6 DOM dan Manipulasi Halaman

DOM menyajikan dokumen sebagai kumpulan node. JavaScript dapat memilih node, membaca nilainya, mengubah konten, mengatur class, atau membuat node baru. Perubahan dilakukan setelah elemen target tersedia.

A. Memilih Elemen

```
const judul = document.querySelector('#judul-utama');
const tombol = document.querySelector('[data-action="ubah"]');
const semuaCard = document.querySelectorAll('.feature-card');
```

querySelector mengembalikan elemen pertama yang cocok atau null. querySelectorAll mengembalikan kumpulan elemen. Periksa selector di panel Elements jika hasilnya tidak sesuai.

B. Mengubah Konten dan Class

```
judul.textContent = 'Belajar JavaScript Dasar';
judul.classList.add('highlight');
judul.classList.toggle('is-active');
```

Gunakan textContent untuk teks biasa. Gunakan classList agar tampilan visual tetap menggunakan CSS. Hindari menyisipkan input pengguna melalui innerHTML karena dapat membentuk markup yang tidak diinginkan.

C. Membuat Elemen

```
function buatCard(item) {
  const article = document.createElement('article');
  const heading = document.createElement('h3');
  const description = document.createElement('p');

  heading.textContent = item.judul;
  description.textContent = item.deskripsi;
  article.classList.add('feature-card');
  article.append(heading, description);
  return article;
}
```

Pertanyaan pemahaman:

- 1) Apa yang terjadi jika querySelector tidak menemukan elemen?
- 2) Mengapa classList lebih tepat untuk perubahan visual?
- 3) Apa keuntungan membuat Card dari data?

2.7 Event dan Interaksi Pengguna

Event adalah pemberitahuan bahwa sesuatu terjadi, misalnya tombol diklik, input berubah, atau form dikirim. addEventListener memasang event dengan function handler tanpa mencampurkan JavaScript ke atribut HTML.

```
const tombolTema = document.querySelector('#toggle-tema');
tombolTema.addEventListener('click', () => {
  document.body.classList.toggle('dark-theme');
});
```

A. Event Object

```
const namaInput = document.querySelector('#nama');
namaInput.addEventListener('input', (event) => {
  console.log(event.target.value);
});
```

B. Mencegah Perilaku Bawaan Form

```
const form = document.querySelector('#form-kontak');

form.addEventListener('submit', (event) => {
  event.preventDefault();
  console.log('Form diperiksa sebelum dikirim');
});
```

Eksperimen 2. DOM dan Event

Pada eksperimen ini mahasiswa menggunakan satu halaman siap pakai. Ketik dan uji JavaScript secara bertahap. Simpan file dan muat ulang halaman setelah setiap perubahan agar hubungan antara aksi pengguna, event, handler, perubahan DOM, dan tampilan browser dapat diamati.

Tujuan Eksperimen

1. Memilih elemen HTML menggunakan `querySelector`.
2. Menangani event click dan input menggunakan `addEventListener`.
3. Mengubah `textContent`, atribut aksesibilitas, dan class CSS melalui JavaScript.
4. Mendiagnosis selector yang tidak cocok dan menangani nilai null secara terarah.

Bahan Awal

Buat folder dengan struktur berikut. Nama folder dan file harus sama agar path CSS dan JavaScript pada `index.html` dapat ditemukan oleh browser.

```
eksperimen-2-dom-event/
|-- index.html
|-- css/
|   |-- style.css
|-- js/
|   |-- app.js
```

File `index.html`

Salin kode berikut ke `index.html`.

```

<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1">
  <title>Eksperimen DOM dan Event</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <main class="container">
    <section class="praktikum-card"
      aria-labelledby="judul-utama">
      <p class="label">Eksperimen 2</p>
      <h1 id="judul-utama">Latihan DOM dan Event</h1>

      <p id="status" aria-live="polite">
        Halaman siap digunakan.
      </p>

      <label for="nama">Nama mahasiswa</label>
      <input id="nama" name="nama" type="text"
        maxlength="30"
        placeholder="ketik nama Anda">
      <p class="counter">
        <span id="jumlah-karakter">0</span>/30 karakter
      </p>

      <div class="actions">
        <button id="ubah-judul" type="button">
          Ubah judul
        </button>
        <button id="toggle-status" type="button"
          aria-pressed="false">
          Toggle status
        </button>
      </div>
    </section>
  </main>

  <script src="js/app.js" defer></script>
</body>
</html>

```

File css/style.css

Salin Bagian 1 dan Bagian 2 secara berurutan ke file css/style.css yang sama.

Bagian 1 - layout dan komponen

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  font-family: Arial, sans-serif;
  color: #1f2937;
  background: #f3f4f6;
  transition: background 200ms, color 200ms;
}

.container {
  width: min(90%, 720px);
  margin-inline: auto;
  padding-block: 64px;
}

.praktikum-card {
  padding: 32px;
  border: 2px solid #d1d5db;
  border-radius: 16px;
  background: #ffffff;
}

.label {
  margin: 0 0 8px;
  font-weight: 700;
  color: #2563eb;
}

label,
input {
  display: block;
  width: 100%;
}

input {
  margin-top: 8px;
  padding: 12px;
  border: 1px solid #9ca3af;
  border-radius: 8px;
  font: inherit;
}

.counter {
  margin-top: 6px;
  color: #6b7280;
}
```

Bagian 2 - tombol dan mode aktif

```
.actions {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  margin-top: 24px;
}

button {
  padding: 10px 16px;
  border: 0;
  border-radius: 8px;
  color: #ffffff;
  background: #2563eb;
  cursor: pointer;
}

button:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}

body.is-active {
  color: #f9fafb;
  background: #111827;
}

body.is-active .praktikum-card {
  border-color: #60a5fa;
  background: #1f2937;
}

body.is-active .counter {
  color: #d1d5db;
}
```

File js/app.js

Salin kode awal berikut ke js/app.js. Bagian pemilihan elemen sudah disediakan; event handler ditambahkan pada langkah kerja.

```
'use strict';

const judulUtama = document.querySelector('#judul-utama');
const status = document.querySelector('#status');
const namaInput = document.querySelector('#nama');
const jumlahKarakter =
  document.querySelector('#jumlah-karakter');
const tombolUbahJudul =
  document.querySelector('#ubah-judul');
const tombolToggleStatus =
  document.querySelector('#toggle-status');

console.log({
  judulUtama,
  status,
  namaInput,
  jumlahKarakter,
  tombolUbahJudul,
  tombolToggleStatus
});
```

Langkah Kerja

Langkah 1. Memilih dan Memeriksa Elemen

Buka index.html melalui browser, lalu buka DevTools pada panel Console. Pastikan keenam nilai yang ditampilkan oleh console.log berupa elemen dan bukan null. Pilih salah satu elemen pada panel Elements dan cocokkan nilai id dengan selector pada app.js.

Catat prediksi sebelum memuat ulang halaman dan tuliskan hasil aktual pada tabel observasi.

Langkah 2. Mengubah Judul melalui Event Click

Tambahkan kode berikut setelah console.log pada app.js.

```
tombolUbahJudul.addEventListener('click', () => {
  judulUtama.textContent = 'DOM Berhasil Diubah';
  status.textContent = 'Teks heading berhasil diubah.';
});
```

Simpan file, muat ulang halaman, lalu klik tombol Ubah judul. Periksa perubahan pada halaman dan panel Elements. Jelaskan mengapa teks berubah tanpa memuat ulang dokumen HTML dari server.

Langkah 3. Melakukan Toggle Class

Tambahkan event handler kedua berikut.

```
tombolToggleStatus.addEventListener('click', () => {
  const aktif = document.body.classList.toggle('is-active');

  tombolToggleStatus.setAttribute(
    'aria-pressed',
    String(aktif)
  );

  status.textContent = aktif
    ? 'Mode aktif dinyalakan.'
    : 'Mode aktif dimatikan.';
});
```

Klik tombol Toggle status beberapa kali. Periksa class pada elemen body dan nilai aria-pressed pada panel Elements. Jelaskan pembagian peran JavaScript dan CSS pada perubahan tampilan tersebut.

Langkah 4. Menangani Event Input

Tambahkan event handler ketiga berikut.

```
namaInput.addEventListener('input', (event) => {
  const jumlah = event.target.value.length;
  jumlahKarakter.textContent = jumlah;
});
```

Ketik nama secara perlahan, hapus beberapa karakter, lalu tempelkan teks. Amati kapan event input dijalankan dan bandingkan nilai event.target.value dengan tampilan penghitung karakter.

Langkah 5. Eksperimen Selector Tidak Cocok

Lakukan kerusakan terkontrol berikut. Jangan langsung memperbaiki kode karena tujuan langkah ini adalah membaca bukti error.

- Pada index.html, ubah sementara id="status" menjadi id="status-pesan" tanpa mengubah selector pada app.js.
- Simpan, muat ulang halaman, lalu klik tombol Ubah judul. Catat pesan error, nama file, dan nomor baris yang tampil pada Console.
- Jalankan console.log(status) pada Console. Pastikan nilainya null, lalu cocokkan selector dengan panel Elements.
- Pulihkan id="status" agar sesuai dengan selector #status dan uji kembali.

Setelah penyebab ditemukan, tambahkan function berikut setelah deklarasi elemen.

```
function ubahStatus(pesan) {
  if (!status) {
    console.warn('Elemen #status tidak ditemukan.');
```

```
    return;
  }

  status.textContent = pesan;
}
```

Ganti setiap baris status.textContent = ... pada event handler dengan pemanggilan ubahStatus(...). Ulangi kerusakan selector. Bandingkan pesan Console sebelum dan sesudah pemeriksaan null, kemudian pulihkan id yang benar.

Alur Event yang Diamati

Urutan yang harus dapat dijelaskan: pengguna melakukan aksi → browser membangkitkan event → addEventListener menjalankan handler → handler membaca atau mengubah DOM dan class → browser merender perubahan tampilan. Console dan panel Elements digunakan sebagai bukti pada setiap tahap.

Tabel Observasi

Isi prediksi sebelum menjalankan setiap langkah. Setelah pengujian, tulis hasil aktual dan bukti yang dapat diperiksa.

Langkah	Prediksi sebelum diuji	Hasil aktual	Bukti atau penjelasan
1 Memilih elemen			
2 Mengubah heading			
3 Toggle class			
4 Event input			
5 Selector tidak cocok			

Kriteria Keberhasilan

1. index.html memuat style.css dan app.js tanpa error path.
2. Semua selector awal menghasilkan elemen dan bukan null.
3. Tombol Ubah judul mengubah heading dan pesan status.
4. Tombol Toggle status menambah atau menghapus class is-active serta memperbarui aria-pressed.
5. Penghitung karakter berubah untuk aktivitas mengetik, menghapus, dan menempelkan teks.
6. Mahasiswa dapat mereproduksi selector tidak cocok, menunjukkan buktinya, dan memperbaikinya.
7. Mahasiswa dapat menjelaskan alur event sampai browser merender perubahan.

Bukti Pengerjaan

1. Tangkapan layar halaman sebelum dan sesudah judul diubah.
2. Tangkapan layar body dengan class is-active pada panel Elements.
3. Tangkapan layar penghitung karakter setelah input diisi.
4. Tangkapan layar Console ketika selector tidak cocok dan setelah perbaikan.
5. Tabel observasi yang telah diisi beserta penjelasan singkat.

Pertanyaan Pemahaman

1. Apa perbedaan peran querySelector dan addEventListener pada eksperimen ini?
2. Mengapa atribut defer digunakan ketika app.js dipanggil dari head?
3. Apa yang dikembalikan querySelector ketika selector tidak menemukan elemen?
4. Mengapa textContent digunakan untuk memperbarui teks pada halaman?
5. Apa hubungan antara event click, function handler, dan perubahan DOM?
6. Mengapa perubahan visual mode aktif ditempatkan pada class CSS, bukan seluruhnya di JavaScript?
7. Apa fungsi event.target pada handler event input?
8. Apa bukti pada DevTools bahwa class is-active benar-benar berubah?
9. Mengapa pemeriksaan null menghasilkan diagnosis yang lebih jelas daripada membiarkan error terjadi?
10. Jelaskan alur lengkap sejak pengguna mengklik tombol sampai tampilan browser berubah.

Independent Challenge 2. FAQ Accordion

Bangun tiga item FAQ dari struktur HTML semantik. Jawaban awal tersembunyi. Ketika tombol pertanyaan diklik, jawaban terkait dibuka dan status aksesibilitas diperbarui.

- Pertanyaan menggunakan elemen button, bukan div yang diberi event klik.
- Gunakan `addEventListener` dan `classList`.
- Nilai `aria-expanded` berubah antara `true` dan `false`.
- Hanya satu jawaban terbuka pada satu waktu.
- Fitur dapat digunakan dengan keyboard melalui perilaku bawaan button.

Larangan: Jangan memakai atribut `onclick` pada HTML dan jangan menyalin library accordion.

2.8 Validasi Form

Validasi sisi klien membantu pengguna memperbaiki data sebelum proses berikutnya. Validasi ini tidak menggantikan validasi server. Pesan harus ditempatkan dekat input, spesifik, dan tidak hanya mengandalkan warna.

A. Membaca dan Membersihkan Input

```
function bacaNilai(selector) {
  const input = document.querySelector(selector);
  return input.value.trim();
}

const nama = bacaNilai('#nama');
const email = bacaNilai('#email');
```

B. Validasi Sederhana

```
function validasiNama(nama) {
  if (nama.length < 3) {
    return 'Nama minimal 3 karakter.';
  }
  return '';
}

function validasiEmail(email) {
  if (!email.includes('@')) {
    return 'Email harus memuat tanda @.';
  }
  return '';
}
```

Pemeriksaan email di atas hanya latihan logika dasar. Untuk implementasi, manfaatkan `type="email"` dan `Constraint Validation API` browser, kemudian tetap lakukan validasi ulang di server.

C. Menampilkan Pesan

```
function tampilkanError(input, elemenError, pesan) {
  elemenError.textContent = pesan;
  input.setAttribute('aria-invalid', String(Boolean(pesan)));
}

form.addEventListener('submit', (event) => {
  event.preventDefault();
  const pesanNama = validasiNama(namaInput.value.trim());
  tampilkanError(namaInput, errorNama, pesanNama);

  if (!pesanNama) {
    statusForm.textContent = 'Data siap diproses.';
  }
});
```

D. Skenario Uji Form

Kasus	Input	Hasil yang Diharapkan
Kosong	nama = ""	Ditolak dengan pesan wajib diisi.

Kasus	Input	Hasil yang Diharapkan
Terlalu pendek	nama = "A"	Ditolak dengan batas minimal.
Valid	nama = "Alya"	Diterima tanpa pesan error.
Email salah	email = "alya.example.com"	Ditolak karena format tidak valid.
Spasi	nama = " Alya "	Spasi tepi dihapus sebelum validasi.

Pertanyaan pemahaman:

- 1) Mengapa trim digunakan?
- 2) Mengapa validasi klien tidak cukup?
- 3) Bagaimana pesan error tetap dapat dipahami tanpa warna?

2.9 Asynchronous JavaScript

JavaScript menjalankan satu call stack utama. Pekerjaan yang memerlukan waktu, seperti timer atau permintaan data, dapat dijadwalkan agar halaman tetap merespons. Asynchronous bukan berarti semua kode berjalan bersamaan; hasilnya tersedia kemudian dan harus ditangani sebagai perubahan state.

A. Sinkron dan Asinkron

Aspek	Sinkron	Asinkron
Urutan	Baris berikut menunggu baris sebelumnya selesai.	Pekerjaan dijadwalkan; hasil diterima kemudian.
Contoh	Perhitungan dan manipulasi array.	Timer, fetch, pembacaan data.
Risiko	Pekerjaan berat dapat memblokir antarmuka.	Race condition dan error yang tidak ditangani.
Bukti	Output sesuai urutan pemanggilan.	Urutan output mengikuti waktu penyelesaian.

```
console.log('A: mulai');
setTimeout(() => console.log('C: timer selesai'), 0);
console.log('B: akhir');

// Prediksi urutan output sebelum menjalankan kode.
```

Walaupun delay bernilai 0, callback timer masuk antrean dan dijalankan setelah call stack kosong. Model mental ini membantu memahami mengapa output tidak selalu mengikuti posisi baris kode.

B. Promise dan Perubahan State

Promise mewakili hasil yang belum tersedia. State awalnya pending, kemudian menjadi fulfilled dengan nilai atau rejected dengan alasan error. Setelah settled, state Promise tidak berubah lagi.

```
function tunggu(ms) {
  return new Promise((resolve) => {
    setTimeout(() => resolve('Selesai setelah ${ms} ms'), ms);
  });
}

tunggu(800)
  .then((pesan) => console.log(pesan))
  .catch((error) => console.error(error));
```

C. async dan await

Function yang diberi async selalu mengembalikan Promise. Kata await menunda langkah di dalam function tersebut sampai Promise selesai, tanpa membekukan seluruh halaman.

```
async function jalankanProses() {
  try {
    console.log('Memulai...');
    const pesan = await tunggu(800);
    console.log(pesan);
  } catch (error) {
    console.error('Proses gagal:', error.message);
  } finally {
    console.log('Proses berakhir');
  }
}

jalankanProses();
```

Pertanyaan pemahaman:

- 1) Apa perbedaan pending, fulfilled, dan rejected?
- 2) Bagian mana yang ditunda oleh await?
- 3) Mengapa finally berguna untuk mereset tombol atau indikator loading?

D. fetch dan Data JSON

fetch mengembalikan Promise berisi Response. Status HTTP 404 atau 500 tidak otomatis masuk catch sehingga response.ok harus diperiksa. Data JSON dibaca melalui response.json yang juga bersifat asinkron.

```
async function ambilFitur() {
  const response = await fetch('data/features.json');

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  return response.json();
}
```

E. State Antarmuka

State	Tampilan	Pemicu
idle	Belum ada proses	Halaman baru dibuka
loading	Indikator dan tombol nonaktif	Permintaan dimulai
success	Data ditampilkan	Data valid diterima
empty	Pesan data belum tersedia	Array hasil kosong
error	Pesan gagal dan tombol coba lagi	Error jaringan, HTTP, atau parsing

```

async function muatFitur() {
  tampilkanState('loading');
  try {
    const data = await ambilFitur();
    if (!Array.isArray(data)) throw new Error('Format data tidak valid');
    if (data.length === 0) return tampilkanState('empty');
    renderFitur(data);
    tampilkanState('success');
  } catch (error) {
    console.error(error);
    tampilkanState('error', error.message);
  }
}

```

Jalankan proyek melalui local development server (via localhost). Membuka index.html langsung melalui file:// dapat membuat fetch file lokal gagal karena kebijakan keamanan browser.

Eksperimen 3. Promise Fetch dan State UI

Pada eksperimen ini mahasiswa membangun pemuat data JSON lokal. Kerjakan secara bertahap melalui local development server agar fetch dapat berjalan. Setiap skenario harus diamati melalui tampilan halaman, Console, dan panel Network.

Tujuan Eksperimen

1. Memprediksi urutan eksekusi sinkron dan asinkron.
2. Menjelaskan state Promise serta menggunakan async dan await.
3. Membaca data JSON menggunakan fetch dan memeriksa response.ok.
4. Menampilkan state idle, loading, success, empty, dan error pada antarmuka.
5. Mendiagnosis kegagalan path, HTTP, parsing JSON, dan klik berulang.

Bahan Awal

Buat folder berikut. Jalankan folder proyek melalui server lokal bawaan IDE atau ekstensi pengembangan web; jangan membuka index.html langsung melalui file://.

```

eksperimen-4-async/
|-- index.html
|-- css/
|   |-- style.css
|-- data/
|   |-- features.json
|-- js/
|   |-- app.js

```

File index.html

Salin kode berikut ke index.html.

```

<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1">
  <title>Eksperimen Asynchronous JavaScript</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <main class="container">
    <section class="praktikum-card"
      aria-labelledby="judul-utama">
      <p class="label">Eksperimen 4</p>
      <h1 id="judul-utama">Async Data Loader</h1>
      <p id="status" class="status" data-state="idle"
        role="status" aria-live="polite">
        Belum ada proses.
      </p>

      <button id="muat-data" type="button">
        Muat data
      </button>

      <section id="daftar-fitur" class="grid"
        aria-label="Daftar fitur"></section>
    </section>
  </main>

  <script src="js/app.js" defer></script>
</body>
</html>

```

File css/style.css

Salin Bagian 1 dan Bagian 2 secara berurutan ke file css/style.css yang sama.

Bagian 1 - layout dan kontrol

```

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  color: #1f2937;
  background: #f3f4f6;
}

.container {
  width: min(90%, 760px);
  margin-inline: auto;
  padding-block: 64px;
}

.praktikum-card {
  padding: 32px;
  border-radius: 16px;
  background: #ffffff;
}

.label {
  font-weight: 700;
  color: #2563eb;
}

button {
  padding: 10px 16px;
  border: 0;
  border-radius: 8px;
  color: #ffffff;
  background: #2563eb;
  cursor: pointer;
}

button:disabled {
  cursor: wait;
  opacity: 0.6;
}

```

Bagian 2 - state dan kartu

```
.status {
  margin-block: 20px;
  padding: 12px;
  border-left: 4px solid #6b7280;
  background: #f9fafb;
}

.status[data-state="loading"] {
  border-color: #d97706;
}

.status[data-state="success"] {
  border-color: #16a34a;
}

.status[data-state="empty"],
.status[data-state="error"] {
  border-color: #dc2626;
}

.grid {
  display: grid;
  gap: 12px;
  margin-top: 24px;
}

.feature-card {
  padding: 16px;
  border: 1px solid #d1d5db;
  border-radius: 10px;
}

.feature-card h2,
.feature-card p {
  margin-block: 0 8px;
}
```

File data/features.json

Salin data berikut ke data/features.json.

```
[
  {
    "judul": "Struktur",
    "deskripsi": "HTML semantik dan mudah dipahami."
  },
  {
    "judul": "Visual",
    "deskripsi": "CSS responsif dan tetap terbaca."
  },
  {
    "judul": "Interaksi",
    "deskripsi": "JavaScript mengubah DOM melalui event."
  }
]
```

File js/app.js

Salin kode awal berikut ke js/app.js. Function untuk state dan render sudah tersedia; proses asynchronous ditambahkan pada langkah kerja.

```
'use strict';

const tombolMuat = document.querySelector('#muat-data');
const status = document.querySelector('#status');
const daftarFitur = document.querySelector('#daftar-fitur');

function tampilkanState(state, pesan) {
  status.dataset.state = state;
  status.textContent = pesan;
}

function buatKartu(item) {
  const article = document.createElement('article');
  const heading = document.createElement('h2');
  const description = document.createElement('p');

  article.classList.add('feature-card');
  heading.textContent = item.judul;
  description.textContent = item.deskripsi;
  article.append(heading, description);
  return article;
}

function renderFitur(items) {
  daftarFitur.textContent = '';
  for (const item of items) {
    daftarFitur.append(buatKartu(item));
  }
}
```

Langkah Kerja

Langkah 1. Memprediksi Urutan Eksekusi

Tambahkan kode berikut sementara di akhir app.js. Tulis prediksi urutan output sebelum memuat halaman.

```
console.log('A: mulai');

setTimeout(() => {
  console.log('C: timer selesai');
}, 0);

console.log('B: akhir');
```

Muat ulang halaman dan catat urutan aktual. Jelaskan mengapa delay 0 tidak membuat C tampil sebelum B, lalu hapus kode sementara tersebut.

Langkah 2. Membuat Promise Sederhana

Tambahkan function berikut setelah renderFitur.

```
function tunggu(ms) {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

async function ujiTunggu() {
  console.log('Promise dimulai');
  await tunggu(800);
  console.log('Promise selesai');
}

ujiTunggu();
```

Amati jeda output pada Console. Jelaskan state Promise sebelum dan setelah resolve dijalankan. Setelah pengamatan, function tunggu boleh dipertahankan untuk simulasi delay.

Langkah 3. Mengambil Data dengan Fetch

Tambahkan function berikut.

```

async function ambilFitur() {
  const response = await fetch('data/features.json');

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  return response.json();
}

```

Buka panel Network, panggil `ambilFitur()` dari Console dengan `await`, lalu periksa request, status HTTP, response, dan nilai array yang dihasilkan.

Langkah 4. Mengelola State Antarmuka

Tambahkan function pemuatan berikut.

```

async function muatFitur() {
  tombolMuat.disabled = true;
  tombolMuat.setAttribute('aria-busy', 'true');
  daftarFitur.textContent = '';
  tampilkanState('loading', 'Memuat data...');

  try {
    const data = await ambilFitur();

    if (!Array.isArray(data)) {
      throw new Error('Format data bukan array.');

```

Klik Muat data dan amati urutan state loading lalu success. Gunakan Network throttling bila perlu agar state loading terlihat lebih lama.

Langkah 5. Eksperimen Kegagalan Terkontrol

Uji satu skenario pada satu waktu. Pulihkan kondisi awal sebelum berpindah skenario.

- Data valid: gunakan `features.json` awal dan buktikan state success serta tiga kartu tampil.
- Data kosong: ganti isi `features.json` menjadi `[]` dan buktikan state empty.
- Path salah: ubah `fetch` menjadi `data/feature.json` dan catat status pada Network serta pesan Console.
- JSON rusak: hapus satu tanda koma atau kurung, lalu bedakan error parsing dari error HTTP.
- Klik berulang: aktifkan throttling dan klik tombol beberapa kali; buktikan atribut `disabled` mencegah request ganda selama loading.

Alur Asynchronous yang Diamati

Urutan yang harus dapat dijelaskan: pengguna mengklik tombol → handler memanggil async function → UI masuk state loading → fetch mengembalikan Promise → `await` menunggu hasil tanpa membekukan halaman → response dan JSON diperiksa → UI masuk state success, empty, atau error → finally mengaktifkan kembali tombol.

Tabel Observasi

Isi prediksi sebelum pengujian, kemudian tuliskan hasil aktual dan bukti dari halaman, Console, atau Network.

Langkah	Prediksi sebelum diuji	Hasil aktual	Bukti atau penjelasan
Urutan A B C			
Data valid			
Array kosong			
Path atau JSON salah			
Klik berulang			

Kriteria Keberhasilan

1. Proyek berjalan melalui local development server dan request JSON terlihat pada Network.
2. Mahasiswa dapat menjelaskan urutan output setTimeout dan perubahan state Promise.
3. fetch memeriksa response.ok dan data hasil parsing harus berupa array.
4. State idle, loading, success, empty, dan error tampil dengan pesan yang jelas.
5. Tombol dinonaktifkan selama loading dan aktif kembali pada finally.
6. Kartu dibentuk menggunakan createElement dan textContent.
7. Mahasiswa dapat membedakan kegagalan HTTP, path, dan parsing JSON berdasarkan bukti.

Bukti Pengerjaan

1. Tangkapan layar Console yang menunjukkan urutan A, B, C.
2. Tangkapan layar state loading dan success dengan kartu data.
3. Tangkapan layar state empty.
4. Tangkapan layar Network dan Console untuk path atau JSON yang salah.
5. Tabel observasi yang telah diisi dan salinan akhir empat file proyek.

Pertanyaan Pemahaman

1. Mengapa callback setTimeout dengan delay 0 tetap dijalankan setelah kode sinkron?
2. Apa perbedaan state pending, fulfilled, dan rejected pada Promise?
3. Mengapa await hanya dapat digunakan pada konteks asynchronous yang sesuai?
4. Mengapa fetch dengan status HTTP 404 tidak otomatis masuk ke catch?
5. Apa fungsi response.ok sebelum response.json dipanggil?
6. Apa perbedaan kegagalan path atau HTTP dengan kegagalan parsing JSON?
7. Mengapa state loading harus selalu diakhiri?
8. Mengapa tombol dinonaktifkan ketika request berlangsung?
9. Mengapa renderFitur menggunakan textContent untuk data JSON?
10. Jelaskan alur sejak tombol diklik sampai UI menampilkan success, empty, atau error.

Independent Challenge 3. Async Data Loader

Bangun komponen pemuat kutipan atau tips belajar dari file JSON lokal. Pengguna menekan tombol Muat Data dan menerima satu item secara acak setelah delay buatan 500–1500 ms.

- Tombol dinonaktifkan dan teks berubah saat loading.
- Data ditampilkan dengan textContent, bukan innerHTML.
- Sediakan state empty, error, dan tombol Coba Lagi.
- Tambahkan opsi simulasi gagal dengan peluang 30%.
- Cegah klik berulang menghasilkan render yang tidak konsisten.
- Catat minimal lima skenario uji dan jelaskan alur Promise.

Larangan: jangan memakai library request, jangan menyalin komponen jadi, dan jangan menyembunyikan error hanya dengan menghapus console.log.

2.10 Debugging dan Troubleshooting

Debugging adalah proses menemukan perbedaan antara perilaku yang diharapkan dan perilaku aktual. Mulailah dari langkah reproduksi, baca pesan pertama pada Console, periksa baris terkait, lalu uji satu perubahan kecil.

Gejala atau Error	Kemungkinan Penyebab	Langkah Diagnosis
Script tidak berjalan	Path salah atau file tidak dimuat	Periksa Network, src, dan console awal.
ReferenceError	Nama variabel atau function tidak tersedia	Bandingkan ejaan dan scope.
TypeError membaca null	Selector tidak menemukan elemen	Uji selector di Console dan periksa urutan script.
Hasil 105, bukan 15	Input masih berupa string	Periksa typeof dan gunakan Number.
Form memuat ulang halaman	preventDefault belum dipanggil	Periksa handler event submit.
Handler berjalan dua kali	Listener terpasang berulang	Gunakan breakpoint atau log saat pemasangan.
Loop tidak berhenti	Kondisi akhir tidak pernah tercapai	Telusuri perubahan counter.

Eksperimen 4. Diagnosis Error JavaScript

Pada eksperimen ini mahasiswa menggunakan kalkulator sederhana yang awalnya bekerja dengan benar. Kerusakan dibuat satu per satu agar hubungan antara gejala, pesan error, nilai aktual, dugaan penyebab, perbaikan, dan uji ulang dapat didokumentasikan.

Tujuan Eksperimen

1. Membuat langkah reproduksi sebelum mengubah kode.
2. Membaca pesan error pertama, nama file, dan nomor baris pada Console.
3. Memeriksa selector, scope, tipe data, event, dan jumlah pemanggilan handler.
4. Menerapkan satu perbaikan kecil dan membuktikannya melalui uji ulang.

Bahan Awal

Buat folder berikut dan salin tiga file starter.

```
eksperimen-3-debugging/
|-- index.html
|-- css/
|   |-- style.css
|-- js/
|   |-- app.js
```

File index.html

Salin kode berikut ke index.html.

```
<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1">
  <title>Eksperimen Diagnosis Error</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <main class="container">
    <section class="praktikum-card"
      aria-labelledby="judul-utama">
      <p class="label">Eksperimen 3</p>
      <h1 id="judul-utama">Kalkulator Total</h1>

      <form id="form-hitung">
        <label for="harga">Harga</label>
        <input id="harga" name="harga" type="number"
          min="1" required>

        <label for="jumlah">Jumlah</label>
        <input id="jumlah" name="jumlah" type="number"
          min="1" required>

        <button id="hitung" type="submit">Hitung</button>
      </form>

      <p>Total: Rp <output id="hasil">0</output></p>
      <p id="pesan" role="status" aria-live="polite">
        Masukkan harga dan jumlah.
      </p>
    </section>
  </main>

  <script src="js/app.js" defer></script>
</body>
</html>
```

File css/style.css

Salin kode berikut ke css/style.css.

```

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  color: #1f2937;
  background: #f3f4f6;
}

.container {
  width: min(90%, 640px);
  margin-inline: auto;
  padding-block: 64px;
}

.praktikum-card {
  padding: 32px;
  border-radius: 16px;
  background: #ffffff;
}

.label {
  font-weight: 700;
  color: #2563eb;
}

label,
input {
  display: block;
  width: 100%;
}

input {
  margin-block: 6px 16px;
  padding: 10px;
}

button {
  padding: 10px 16px;
  border: 0;
  border-radius: 8px;
  color: #ffffff;
  background: #2563eb;
}

```

File js/app.js

Salin kode berikut ke js/app.js. Ini adalah baseline yang harus bekerja sebelum kerusakan dibuat.

```
'use strict';

const form = document.querySelector('#form-hitung');
const hargaInput = document.querySelector('#harga');
const jumlahInput = document.querySelector('#jumlah');
const hasil = document.querySelector('#hasil');
const pesan = document.querySelector('#pesan');

function hitungTotal(harga, jumlah) {
  return harga * jumlah;
}

function tampilkanPesan(teks) {
  pesan.textContent = teks;
}

function prosesForm(event) {
  console.count('prosesForm');
  event.preventDefault();

  const harga = Number(hargaInput.value);
  const jumlah = Number(jumlahInput.value);

  if (harga <= 0 || jumlah <= 0) {
    tampilkanPesan('Harga dan jumlah harus positif.');
```

Langkah Kerja

Langkah 1. Memverifikasi Baseline

Buka halaman, masukkan harga 10000 dan jumlah 2, lalu submit. Pastikan total Rp20.000, pesan keberhasilan tampil, dan console.count bertambah satu. Simpan salinan baseline sebelum membuat kerusakan.

Langkah 2. Kasus A Selector Tidak Cocok

1. Pada index.html, ubah sementara id="form-hitung" menjadi id="form-total" tanpa mengubah app.js.
2. Muat ulang halaman dan baca error pertama pada Console. Catat nama file dan nomor baris.
3. Jalankan console.log(form) dan buktikan nilainya null. Cocokkan id pada panel Elements dengan selector pada app.js.
4. Pulihkan id yang benar, muat ulang, dan ulangi uji baseline.

Langkah 3. Kasus B Nama Variabel Salah

Ubah sementara hargaInput.value menjadi hargainput.value di dalam prosesForm. Submit form, catat jenis error dan barisnya, lalu bandingkan kapitalisasi nama variabel. Pulihkan penulisan hargaInput dan uji ulang.

Langkah 4. Kasus C Perhitungan Menjadi Teks

Lakukan dua perubahan sementara berikut, lalu masukkan harga 10 dan jumlah 5.

```
// Hilangkan konversi Number untuk eksperimen.
const harga = hargaInput.value;
const jumlah = jumlahInput.value;

// Ubah operator di hitungTotal.
return harga + jumlah;
```

Prediksi hasil sebelum submit. Gunakan breakpoint atau console.log(typeof harga, typeof jumlah) untuk membuktikan tipe aktual. Pulihkan Number dan operator perkalian, kemudian uji ulang.

Langkah 5 Kasus D Form Memuat Ulang Halaman

Aktifkan Preserve log pada Console, lalu komentari `event.preventDefault()`. Submit form dan amati reload, perubahan URL bila ada, nilai input, serta urutan log. Pulihkan `preventDefault` dan buktikan halaman tidak memuat ulang.

Langkah 6 Kasus E Handler Berjalan Dua Kali

Tambahkan listener kedua berikut setelah listener baseline, lalu submit satu kali.

```
form.addEventListener('submit', (event) => {
  prosesForm(event);
});
```

Periksa `console.count` dan buktikan `prosesForm` dipanggil dua kali untuk satu submit. Hapus listener tambahan, muat ulang, lalu pastikan hitungan kembali bertambah satu per submit.

Metode Diagnosis yang Digunakan

Urutan diagnosis: tulis gejala dan ekspektasi → reproduksi secara konsisten → baca error pertama → periksa nilai atau selector pada baris terkait → buat satu dugaan → lakukan satu perubahan kecil → ulangi kasus awal dan satu kasus tambahan. Jangan menghapus pesan error sebelum sumbernya dipahami.

Tabel Observasi

Isi prediksi sebelum menjalankan kerusakan. Bukti dapat berupa pesan error, nilai variabel, selector, `console.count`, atau perilaku reload.

Langkah	Prediksi sebelum diuji	Hasil aktual	Bukti atau penjelasan
Selector			
Variabel			
Tipe data			
Reload form			
Handler ganda			

Kriteria Keberhasilan

1. Baseline menghasilkan total yang benar tanpa error Console.
2. Setiap kerusakan direproduksi satu per satu dan kondisi awal dipulihkan sebelum kasus berikutnya.
3. Catatan memuat gejala, ekspektasi, error atau nilai aktual, dugaan, perbaikan, dan uji ulang.
4. Mahasiswa dapat membedakan `ReferenceError`, `TypeError` akibat null, kesalahan logika atau tipe, reload form, dan handler ganda.
5. Satu perbaikan kecil terbukti menyelesaikan kasus tanpa merusak baseline.

Bukti Pengerjaan

1. Tangkapan layar baseline yang benar.
2. Tangkapan layar Console untuk kasus selector dan nama variabel.
3. Bukti `typeof` untuk kasus perhitungan menjadi teks.
4. Bukti reload form dan `console.count` pada handler ganda.
5. Tabel observasi serta salinan akhir tiga file yang sudah kembali benar.

Pertanyaan Pemahaman

1. Mengapa error pertama pada Console diperiksa sebelum error berikutnya?
2. Apa perbedaan gejala, penyebab, dugaan, dan bukti?
3. Mengapa selector yang tidak cocok dapat menghasilkan nilai null?
4. Mengapa JavaScript membedakan `hargaInput` dan `hargainput`?
5. Mengapa input bernilai 10 dan 5 dapat menghasilkan teks 105?

6. Bagaimana typeOf membantu diagnosis kesalahan perhitungan?
7. Apa fungsi preventDefault pada event submit?
8. Bagaimana console.count membuktikan handler berjalan lebih dari sekali?
9. Mengapa hanya satu perubahan dilakukan pada setiap percobaan perbaikan?
10. Apa bukti minimum bahwa perbaikan benar-benar berhasil dan bukan hanya menyembunyikan gejala?

2.11 Integrasi dan Checklist Kualitas

Pisahkan struktur folder, data, logika, dan akses DOM secara wajar. Untuk skala latihan, satu file app.js diperbolehkan selama urutannya jelas dan setiap function memiliki satu tanggung jawab.

```
praktikum-modul-02_nim_nama/
|-- index.html
|-- css/style.css
|-- js/app.js
|-- data/features.json
|-- assets/
|-- README.md
|-- worksheet.pdf
```

- ☐ File JavaScript terhubung dengan defer dan tidak menghasilkan error saat halaman dibuka.
- ☐ Nama variabel dan function menjelaskan isi atau tindakan.
- ☐ Input dikonversi dan divalidasi sebelum digunakan.
- ☐ Function logika mengembalikan nilai dan dapat diuji terpisah dari DOM.
- ☐ Selector disimpan pada variabel dan sesuai dengan HTML.
- ☐ Interaksi menggunakan addEventListener, bukan atribut event inline.
- ☐ Perubahan visual memakai class CSS.
- ☐ Button dan form tetap dapat digunakan dengan keyboard.
- ☐ Pesan error spesifik, dekat dengan input, dan tidak hanya mengandalkan warna.
- ☐ Proses asynchronous memiliki state idle, loading, success, empty, dan error yang jelas.
- ☐ fetch memeriksa response.ok serta kegagalan HTTP atau parsing dicatat dan ditangani.
- ☐ Tidak ada pesan error yang belum dijelaskan pada Console.

3. Tugas Praktikum

Task dikerjakan berurutan dalam satu folder proyek. Task 1 membangun logika dasar. Task 2 menggabungkan data, DOM, event, dan validasi form. Task 3 menerapkan Asynchronous JavaScript untuk memuat data JSON. Minimal 70% fungsi harus selesai pada sesi praktikum; mini project belum menggunakan asynchronous.

Alokasi waktu yang disarankan: Task 1 selama 50 menit, Task 2 selama 70 menit, Task 3 selama 80 menit, serta pengujian.

Komponen	Bobot	Hasil Utama
Task 1 Logic Builder	20%	Function pengolahan nilai dan kasus uji.
Task 2 DOM dan Form Controller	25%	Daftar peserta dirender, difilter, dan ditambah melalui form tervalidasi.
Task 3 Async Data Loader	25%	Data JSON dimuat dan seluruh state proses ditangani.
Homework	30%	Interactive Profile Card teruji.

Task 1. JavaScript Logic Builder

Target: membangun dan menguji logika pengolahan nilai melalui function murni tanpa DOM. Waktu yang disarankan 50 menit.

Skenario dan Aturan

1. Program menerima nama mahasiswa dan nilai akhir berupa number.
2. Nilai valid berada pada rentang 0 sampai 100 dan harus berupa angka finite.
3. Kategori A untuk 85–100, B untuk 70–84, C untuk 60–69, dan D untuk 0–59.
4. Status Lulus diberikan untuk nilai minimal 60; selain itu Tidak lulus.
5. Input tidak valid menghasilkan kategori null dan status Data tidak valid.

Bahan Awal

Buat folder berikut dan salin starter app.js. Jalankan melalui Console browser atau Node.js.

```
task-1-logic-builder/
-- app.js
```

File app.js

```
'use strict';

function validasiNilai(nilai) {
  // TODO: return true hanya untuk number finite pada 0-100.
}

function tentukanKategori(nilai) {
  // TODO: tangani nilai tidak valid, lalu kembalikan A/B/C/D.
}

function tentukanStatus(nilai) {
  // TODO: kembalikan Data tidak valid, Lulus, atau Tidak lulus.
}

function buatRingkasan(nama, nilai) {
  // TODO: return object berisi nama, nilai, kategori, dan status.
}

const kasusUji = [
  { nama: 'Alya', nilai: 0 },
  { nama: 'Bima', nilai: 59 },
  { nama: 'Citra', nilai: 60 },
  { nama: 'Danu', nilai: 69 },
  { nama: 'Eka', nilai: 70 },
  { nama: 'Fani', nilai: 85 },
  { nama: 'Gilang', nilai: 101 },
];

const hasilUji = kasusUji.map(({ nama, nilai }) =>
  buatRingkasan(nama, nilai)
);

console.table(hasilUji);
```

Langkah Kerja

1. Jalankan starter. Amati bahwa hasil belum benar karena empat function masih berisi TODO.
2. Implementasikan validasiNilai. Uji terpisah dengan console.log untuk nilai -1, 0, 100, 101, NaN, dan teks '80'.
3. Implementasikan tentukanKategori. Susun kondisi dari batas tertinggi ke terendah atau gunakan rentang yang tidak tumpang tindih.
4. Implementasikan tentukanStatus dan pastikan nilai tidak valid diperiksa sebelum aturan kelulusan.
5. Implementasikan buatRingkasan tanpa menulis ulang aturan kategori maupun status.
6. Jalankan tujuh kasus uji bawaan, bandingkan hasil aktual dengan tabel acuan, lalu tambahkan sedikitnya dua kasus uji buatan sendiri; salah satunya harus menguji nilai 84.
7. Ubah sementara satu batas kategori saat pengerjaan, prediksi dampaknya, jalankan ulang, kemudian pulihkan aturan awal.

Acuan Hasil Tujuh Kasus Uji

Nilai	Kategori	Status	Catatan
0	D	Tidak lulus	Batas bawah valid
59	D	Tidak lulus	Satu nilai sebelum lulus
60	C	Lulus	Batas kelulusan
69	C	Lulus	Batas atas C
70	B	Lulus	Batas bawah B
85	A	Lulus	Batas bawah A
101	null	Data tidak valid	Di luar rentang

Hint

Gunakan hint secara berurutan. Buka level berikutnya hanya setelah mencoba dan mencatat percobaan sebelumnya pada worksheet.

1. Hint 1: Pecah masalah menjadi empat pertanyaan: valid atau tidak, kategorinya apa, statusnya apa, dan bagaimana hasil digabungkan.
2. Hint 2: Pada kategori, periksa batas 85 lebih dahulu, kemudian 70, kemudian 60. Jika urutan dibalik, kondisi yang terlalu umum dapat menangkap nilai lebih awal.
3. Hint 3: validasiNilai dapat menggabungkan `typeof`, `Number.isFinite`, nilai `>= 0`, dan nilai `<= 100`. Gunakan guard clause pada function lain untuk nilai tidak valid.

Keluaran dan Bukti Wajib

- a. File `app.js` tanpa `TODO` yang tersisa pada empat function inti.
- b. Tabel pada worksheet berisi input, hasil yang diharapkan, hasil aktual, dan status lulus/gagal untuk minimal sembilan kasus.
- c. Tangkapan layar `console.table` dan satu tangkapan layar uji input tidak valid.
- d. Penjelasan singkat mengapa 59, 60, 69, 70, 84, dan 85 termasuk boundary value penting.

Kriteria Selesai

- a. Semua kasus acuan menghasilkan kategori dan status yang tepat.
- b. Setiap function memiliki return value yang jelas dan tidak bergantung pada DOM.
- c. Aturan kategori dan status tidak diduplikasi di `buatRingkasan`.
- d. Mahasiswa mampu menjelaskan alur satu input dan mengubah satu batas nilai.

Pertanyaan Pemahaman

1. Mengapa nilai tepat pada batas kategori harus diuji?
2. Apa perbedaan `return` dan `console.log` dalam sebuah function?
3. Mengapa input teks '80' tidak langsung dianggap nilai yang valid pada task ini?
4. Apa akibat memeriksa kondisi nilai `>= 60` sebelum nilai `>= 85`?

Task 2. DOM dan Form Controller

Target: menampilkan data peserta dari array, menambahkan peserta melalui form tervalidasi, dan memfilter daftar tanpa memuat ulang halaman. Waktu yang disarankan 70 menit.

Bahan Awal

```
task-2-dom-form/
|-- index.html
|-- css/
|   |-- style.css
|-- js/
|   |-- app.js
```

File index.html

```

<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Task 2 DOM dan Form</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <main class="container">
    <h1>Daftar Peserta</h1>

    <form id="form-peserta" novalidate>
      <label for="nama">Nama peserta</label>
      <input id="nama" name="nama" autocomplete="name"
        aria-describedby="error-nama" required>
      <small id="error-nama" class="error"></small>

      <label for="prodi">Program studi</label>
      <select id="prodi" name="prodi"
        aria-describedby="error-prodi" required>
        <option value="">Pilih program studi</option>
        <option>Teknik Informatika</option>
        <option>Sistem Informasi</option>
      </select>
      <small id="error-prodi" class="error"></small>

      <button type="submit">Tambah peserta</button>
    </form>

    <label for="filter-prodi">Filter program studi</label>
    <select id="filter-prodi">
      <option value="semua">Semua</option>
      <option>Teknik Informatika</option>
      <option>Sistem Informasi</option>
      <option>Manajemen Informatika</option>
    </select>

    <p id="status" role="status" aria-live="polite"></p>
    <section id="daftar-peserta" class="grid"
      aria-label="Daftar peserta"></section>
  </main>
  <script src="js/app.js" defer></script>
</body>
</html>

```

File css/style.css

```

* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: Arial, sans-serif;
  color: #1f2937;
  background: #f3f4f6;
}
.container {
  width: min(92%, 760px);
  margin-inline: auto;
  padding-block: 40px;
}
form, .kartu {
  margin-block: 16px;
  padding: 20px;
  border-radius: 12px;
  background: #ffffff;
}
label, input, select, button { display: block; }
input, select, button {
  width: 100%;
  margin-block: 6px 12px;
  padding: 10px;
}
button { cursor: pointer; }
.grid { display: grid; gap: 12px; }
.error { display: block; min-height: 20px; color: #b91c1c; }
[aria-invalid="true"] { outline: 2px solid #b91c1c; }

```

File js/app.js

Starter sudah menyediakan data dan selector. Selesaikan bagian TODO; jangan mengganti data input dengan innerHTML.

```
'use strict';

const peserta = [
  { id: 1, nama: 'Alya', prodi: 'Teknik Informatika' },
  { id: 2, nama: 'Bima', prodi: 'Sistem Informasi' },
];

const form = document.querySelector('#form-peserta');
const namaInput = document.querySelector('#nama');
const prodiInput = document.querySelector('#prodi');
const filterInput = document.querySelector('#filter-prodi');
const daftar = document.querySelector('#daftar-peserta');
const status = document.querySelector('#status');
const errorNama = document.querySelector('#error-nama');
const errorProdi = document.querySelector('#error-prodi');

function validasiPeserta(calon) {
  // TODO: return object { valid, errorNama, errorProdi }.
}

function buatKartuPeserta(item) {
  // TODO: buat article, h2, dan p dengan createElement.
  // Isi teks dengan.textContent, lalu return article.
}

function renderPeserta(data) {
  // TODO: kosongkan daftar, tangani data kosong, lalu append kartu.
}

form.addEventListener('submit', (event) => {
  event.preventDefault();
  // TODO: baca nilai, validasi, atur aria-invalid dan pesan error.
  // Jika valid, push object baru dengan id unik, reset, dan render.
});

filterInput.addEventListener('change', () => {
  // TODO: jika 'semua' gunakan seluruh peserta; selain itu filter.
});

renderPeserta(peserta);
```

Langkah Kerja

1. Buka index.html dan pastikan dua peserta awal belum tampil karena function render masih TODO. Periksa Console dan pastikan tidak ada selector bernilai null.
2. Implementasikan buatKartuPeserta menggunakan createElement, .textContent, .classList.add('kartu'), dan return article.
3. Implementasikan renderPeserta. Gunakan replaceChildren untuk mencegah duplikasi dan tampilkan pesan 'Tidak ada peserta' jika array hasil kosong.
4. Implementasikan validasiPeserta: nama minimal 3 karakter setelah trim dan program studi wajib dipilih.
5. Lengkapi submit handler. Atur aria-invalid serta pesan error, lalu hentikan proses jika tidak valid.
6. Jika valid, buat object dengan id unik, tambahkan ke array, reset form, bersihkan error, dan render ulang.
7. Lengkapi filter change. Filter tidak boleh menghapus isi array peserta.
8. Jalankan seluruh skenario uji. Perbaiki satu masalah pada satu waktu dan ulangi skenario sebelumnya setelah perubahan.

Skenario Uji Wajib

Skenario	Tindakan	Hasil yang diharapkan	Bukti
Awal	Muat halaman	Dua kartu peserta tampil	Screenshot
Kosong	Submit tanpa isi	Dua pesan error; data tidak bertambah	Screenshot + jumlah data

Skenario	Tindakan	Hasil yang diharapkan	Bukti
Nama pendek	Nama 2 karakter	Error hanya pada nama	Screenshot
Valid	Isi semua field	Satu kartu baru tampil	Screenshot
Submit kedua	Tambah peserta lain	Data lama tidak tergandakan	Jumlah kartu
Filter ada	Pilih satu prodi	Hanya prodi tersebut tampil	Screenshot
Filter kosong	Pilih Manajemen Informatika	Pesan Tidak ada peserta	Screenshot
Keyboard	Tab lalu Enter untuk submit	Seluruh kontrol dapat digunakan	Catatan uji

Hint

Gunakan hint secara berurutan. Buka level berikutnya hanya setelah mencoba dan mencatat percobaan sebelumnya pada worksheet.

1. Hint 1: Kerjakan dalam urutan render kartu, render daftar, validasi, submit, lalu filter. Pastikan satu tahap bekerja sebelum lanjut.
2. Hint 2: Untuk render, buat DocumentFragment atau append setiap article. Untuk filter, gunakan peserta.filter tanpa mengubah array asli.
3. Hint 3: Hasil validasi dapat berbentuk { valid: false, errorNama: '...', errorProdi: '...' }. ID unik sederhana dapat memakai Date.now() untuk latihan ini.

Keluaran dan Bukti Wajib

- a. index.html, css/style.css, dan js/app.js yang dapat dijalankan tanpa error Console.
- b. Tabel delapan skenario uji pada worksheet.
- c. Screenshot kondisi awal, validasi gagal, data berhasil ditambahkan, filter berisi hasil, dan filter kosong.
- d. Catatan singkat cara program mencegah render ganda dan perubahan array saat filter.

Kriteria Selesai

- a. Menggunakan createElement, textContent, classList, addEventListener, dan preventDefault.
- b. Validasi dipisahkan dari render DOM dan aria-invalid diperbarui sesuai kondisi.
- c. Submit valid menambah tepat satu object; submit tidak valid tidak mengubah data.
- d. Filter bekerja, pesan empty tersedia, serta halaman dapat digunakan dengan keyboard.
- e. Task ini tidak menggunakan fetch atau async/await.

Pertanyaan Pemahaman

1. Mengapa submit form memerlukan preventDefault?
2. Mengapa validasi sebaiknya dipisahkan dari render DOM?
3. Bagaimana replaceChildren membantu mencegah duplikasi tampilan?
4. Mengapa textContent lebih aman untuk menampilkan data input pengguna?
5. Mengapa proses filter tidak boleh menimpa array peserta asli?

Task 3 Asynchronous JavaScript Data Loader

Target: memuat materi dari JSON lokal melalui fetch dan menampilkan state idle, loading, success, empty, serta error. Waktu yang disarankan 80 menit. Jalankan melalui local development server.

Bahan Awal

```
task-3-async-loader/
|-- index.html
|-- css/
|   |-- style.css
|-- data/
|   |-- materi.json
|-- js/
|   |-- app.js
```

File index.html

```

<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Task 3 Async Data Loader</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <main class="container">
    <h1>Materi JavaScript</h1>
    <p id="status" role="status" aria-live="polite">
      Tekan tombol untuk memuat data.
    </p>
    <button id="muat" type="button">Muat data</button>
    <button id="coba-lagi" type="button" hidden>Coba lagi</button>
    <section id="daftar-materi" class="grid"
      aria-label="Daftar materi"></section>
  </main>
  <script src="js/app.js" defer></script>
</body>
</html>

```

File css/style.css

```

* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: Arial, sans-serif;
  color: #1f2937;
  background: #f3f4f6;
}
.container {
  width: min(92%, 760px);
  margin-inline: auto;
  padding-block: 48px;
}
button { margin: 8px 8px 16px 0; padding: 10px 16px; }
.grid { display: grid; gap: 12px; }
.kartu {
  padding: 16px;
  border-radius: 12px;
  background: #ffffff;
}
#status[data-state="loading"] { color: #1d4ed8; }
#status[data-state="error"] { color: #b91c1c; }
#status[data-state="success"] { color: #166534; }

```

File data/materi.json

```

[
  {
    "id": 1,
    "judul": "DOM Dasar",
    "durasi": 45
  },
  {
    "id": 2,
    "judul": "Event dan Form",
    "durasi": 60
  },
  {
    "id": 3,
    "judul": "Asynchronous JavaScript",
    "durasi": 75
  }
]

```

File js/app.js

```
'use strict';

const status = document.querySelector('#status');
const daftar = document.querySelector('#daftar-materi');
const tombolMuat = document.querySelector('#muat');
const tombolCobaLagi = document.querySelector('#coba-lagi');

function aturState(state, pesan) {
  status.dataset.state = state;
  status.textContent = pesan;
  tombolCobaLagi.hidden = state !== 'error';
}

async function ambilMateri() {
  // TODO: fetch data/materi.json.
  // TODO: jika response.ok false, throw Error yang informatif.
  // TODO: return hasil response.json().
}

function renderMateri(data) {
  // TODO: kosongkan daftar dan buat kartu dengan createElement.
}

async function muatData() {
  aturState('loading', 'Memuat data...');
  tombolMuat.disabled = true;
  daftar.replaceChildren();

  try {
    // TODO: await ambilMateri().
    // TODO: bedakan array kosong dan data berisi.
  } catch (error) {
    console.error(error);
    // TODO: tampilkan state error dan pesan yang dapat dipahami.
  } finally {
    // TODO: aktifkan kembali tombol Muat data.
  }
}

tombolMuat.addEventListener('click', muatData);
tombolCobaLagi.addEventListener('click', muatData);
```

Langkah Kerja

1. Jalankan folder melalui server lokal. Pastikan state awal tampil dan Network tidak menunjukkan permintaan sebelum tombol diklik.
2. Implementasikan ambilMateri dengan fetch, pemeriksaan response.ok, dan response.json.
3. Implementasikan renderMateri menggunakan createElement dan textContent untuk judul serta durasi.
4. Lengkapi try pada muatData. Jika data berisi, render kartu dan tampilkan success; jika array kosong, tampilkan empty.
5. Lengkapi catch dan finally. Pesan error terlihat pada UI, detail error tetap dicatat pada Console, dan tombol selalu aktif kembali.
6. Klik Muat data dua kali dengan cepat. Pastikan disabled mencegah dua proses bersamaan dan daftar tidak terganggu.
7. Lakukan empat eksperimen kegagalan satu per satu: array kosong, path salah, JSON rusak, dan server dimatikan. Pulihkan file setelah setiap eksperimen.
8. Periksa Network untuk URL, status, dan response; lengkapi tabel uji serta screenshot setiap state.

Skenario Uji Wajib

Skenario	Cara membuat	State/hasil yang diharapkan	Bukti
Idle	Muat halaman	Belum ada request dan tombol aktif	UI + Network
Loading	Klik Muat data	Pesan loading; tombol disabled	Screenshot
Success	JSON valid dan berisi	Tiga kartu tampil	UI + status 200
Empty	Ubah JSON menjadi []	Pesan data kosong; bukan error	Screenshot

Skenario	Cara membuat	State/hasil yang diharapkan	Bukti
Path salah	Ubah URL fetch sementara	State error dan status 404	Console + Network
JSON rusak	Hapus satu tanda kutip/koma	Parsing gagal dan state error	Console
Server mati	Hentikan local server	Fetch ditolak dan UI error	Console
Klik ganda	Klik cepat dua kali	Satu proses; daftar tidak ganda	Jumlah request/kartu

Hint

Gunakan hint secara berurutan. Buka level berikutnya hanya setelah mencoba dan mencatat percobaan sebelumnya pada worksheet.

- Hint 1: Pisahkan tanggung jawab: ambilMateri mengambil data, renderMateri membentuk DOM, dan muatData mengatur alur serta state.
- Hint 2: fetch hanya reject pada kegagalan jaringan. Respons 404 tetap harus diperiksa melalui response.ok dan diubah menjadi Error secara manual.
- Hint 3: Pola utama: const data = await ambilMateri(); lalu cek data.length. Pada finally, set tombolMuat.disabled = false agar dijalankan baik saat sukses maupun gagal.

Keluaran dan Bukti Wajib

- index.html, css/style.css, js/app.js, dan data/materi.json yang sudah dipulihkan ke kondisi valid.
- Tabel delapan skenario uji pada worksheet.
- Screenshot state idle, loading, success, empty, dan error.
- Bukti Console dan Network untuk path salah serta JSON rusak.
- Catatan urutan state dan alasan tombol dinonaktifkan selama request.

Kriteria Selesai

- Menggunakan async/await, try/catch/finally, fetch, response.ok, dan response.json.
- State idle, loading, success, empty, dan error dapat dibedakan pada UI.
- Tombol Coba lagi memanggil function pemuatan yang sama dan hanya tampil saat error.
- Klik ganda tidak menghasilkan request atau kartu yang tidak konsisten.
- Tidak ada error yang disembunyikan; bukti Console dan Network dapat dijelaskan saat uji coba program.

Pertanyaan Pemahaman

- Mengapa await digunakan di dalam async function?
- Mengapa response.ok tetap perlu diperiksa setelah fetch berhasil resolve?
- Apa perbedaan state empty dan error dari sudut pandang pengguna?
- Mengapa aktivasi tombol ditempatkan di finally?
- Bagaimana disabled dan replaceChildren mencegah hasil klik ganda yang tidak konsisten?

Homework

Buat Interactive Profile Card. Data profil dan daftar keterampilan berasal dari data/profile.json dan dimuat secara asinkron. Pengguna dapat membuka detail, mengganti tema, menambah keterampilan, serta menghapus keterampilan.

- Profil dan keterampilan awal berasal dari file JSON lokal.
- Tersedia state loading, empty, error, dan Coba Lagi.
- Tombol detail menggunakan classList.toggle dan aria-expanded.
- Form tambah keterampilan memvalidasi input kosong.
- Setiap item memiliki tombol hapus.
- Klik berulang tidak menggandakan data.
- README menjelaskan cara menjalankan melalui local server.

Skenario	Ekspektasi
Halaman dibuka	Loading lalu profil tampil tanpa error.
Path JSON salah	Pesan error dan Coba Lagi tampil.
JSON kosong	State empty tampil.
Detail diklik dua kali	Terbuka lalu tertutup.
Input kosong	Data tidak ditambah; pesan muncul.
Skill valid	Item baru muncul satu kali.
Semua skill dihapus	Pesan kondisi kosong tampil.

Pengumpulan: worksheet PDF, minimal tujuh screenshot pengujian, dan README.

4. Mini Project : Landing Page Interaktif

4.1 Brief Produk

Kembangkan landing page statis Modul 1 menjadi landing page interaktif untuk produk, kegiatan, layanan kampus, atau portofolio. Struktur dan tampilan yang sudah baik dipertahankan, kemudian perilaku ditambahkan secara bertahap melalui JavaScript.

4.2 Fitur Minimal

1. Navigasi mobile yang dapat dibuka dan ditutup melalui button.
2. Daftar fitur atau layanan yang dirender dari array of objects.
3. Filter sederhana terhadap daftar fitur atau layanan.
4. FAQ accordion dengan aria-expanded yang sesuai.
5. Form kontak dengan validasi dan pesan keberhasilan.
6. Tombol kembali ke atas atau interaksi sejenis yang relevan.
7. Satu perubahan visual berbasis class, misalnya tema gelap.

4.3 Batas Teknis

1. Menggunakan HTML, CSS, dan vanilla JavaScript tanpa framework atau library UI.
2. JavaScript berada pada file eksternal dan dipanggil dengan defer.
3. Mini project belum menggunakan fetch, Promise, atau async/await; materi tersebut dinilai melalui Task 3 dan homework.
4. Tidak menggunakan inline event seperti onclick.
5. Tidak menggunakan innerHTML untuk memasukkan data pengguna.
6. Semua fitur utama dapat digunakan dengan keyboard.
7. Tidak ada error yang tersisa pada Console.

4.4 Tahapan Pengerjaan

1. Audit landing page Modul 1 dan pilih elemen yang membutuhkan interaksi.
2. Tuliskan acceptance criteria untuk setiap fitur.
3. Siapkan selector dan data awal.
4. Bangun satu fitur, uji, lalu commit atau simpan versi sebelum berpindah fitur.
5. Uji pada layar kecil dan desktop serta menggunakan keyboard.
6. Perbaiki error, rapikan nama, dan hapus log sementara yang tidak diperlukan.
7. Siapkan demonstrasi singkat dan jawaban saat uji coba.

4.5 Definition of Done

- ☐ Seluruh fitur minimal bekerja sesuai acceptance criteria.
- ☐ Tampilan dasar dari Modul 1 tidak rusak pada viewport 320px, 768px, dan 1024px.
- ☐ Tidak ada error Console pada alur normal.
- ☐ Form menolak input tidak valid dan menerima input valid.
- ☐ Interaksi dapat digunakan dengan mouse dan keyboard.
- ☐ Kode tersusun dalam function kecil dengan penamaan yang jelas.
- ☐ Folder proyek, worksheet, README, dan bukti uji lengkap.
- ☐ Mahasiswa mampu menjelaskan alur event sampai perubahan DOM.

4.6 Acceptance Criteria

Fitur	Kondisi Diterima
Navigasi mobile	Button mengubah status terbuka dan aria-expanded; menu dapat ditutup kembali.
Daftar dinamis	Jumlah kartu mengikuti data dan tidak terduplikasi saat render ulang.
Filter	Pilihan kategori menampilkan subset yang tepat serta menangani kondisi kosong.
FAQ	Jawaban terkait terbuka, jawaban lain tertutup, dan keyboard berfungsi.
Form	Semua aturan validasi diuji; pesan jelas dan ringkasan hanya tampil saat valid.
Tema atau state	Class berubah dan kontras teks tetap terbaca.
Kualitas	Tidak ada error Console, selector null, atau reload form yang tidak disengaja.

4.7 Batas Penggunaan AI

Fase	Tanpa AI	Bantuan yang Diizinkan
Memahami brief	Wajib membaca dan menulis acceptance criteria sendiri.	Meminta penjelasan istilah setelah mencoba merumuskan.
Menulis versi awal	Wajib membuat struktur dan function awal sendiri.	Tidak ada generasi kode lengkap.
Debugging	Wajib membaca error, baris, dan nilai aktual.	Meminta hint diagnosis satu langkah pada satu waktu.
Penyempurnaan	Wajib menguji dan menentukan keputusan akhir.	Meminta review keterbacaan atau edge case.
Coding	Wajib menjelaskan serta mengubah kode sendiri.	Tidak ada bantuan AI.

5. Penilaian dan Refleksi

5.1 Kuesioner Self Assessment

Gunakan skala 1 sampai 5: 1 belum mampu, 2 mampu dengan banyak bantuan, 3 mampu dengan panduan, 4 mandiri, dan 5 mampu menjelaskan kepada orang lain.

Pernyataan	Skor 1 sampai 5
Saya dapat menggunakan tipe data, kondisi, dan function.	
Saya dapat mengolah array of objects.	
Saya dapat memilih dan mengubah elemen DOM.	
Saya dapat memasang event listener dan menjelaskan alurnya.	
Saya dapat memvalidasi form dan menampilkan pesan.	
Saya dapat menjelaskan sinkron dan asinkron.	
Saya dapat menggunakan Promise dan async/await.	
Saya dapat memakai fetch dan memeriksa response.ok.	
Saya dapat menangani loading, success, empty, dan error.	
Saya dapat membaca serta memperbaiki error JavaScript.	
Saya dapat menjelaskan seluruh kode yang dikumpulkan.	

5.2 Peer Assessment

Seorang teman menguji mini project selama 10 sampai 15 menit dan memberikan skor 1 sampai 5 serta satu catatan yang dapat ditindaklanjuti.

Kriteria	Fokus Pemeriksaan
Kesesuaian fitur	Seluruh fitur minimal tersedia.
Daftar dinamis	Data array, render, filter, dan kondisi kosong bekerja.
Interaksi	State konsisten dan dapat diulang.
Validasi	Input salah ditangani dan pesan membantu.
Keyboard	Navigasi, FAQ, dan form dapat digunakan tanpa mouse.
Stabilitas	Tidak ada error atau duplikasi elemen.

5.3 Lesson Learnt

1. Konsep JavaScript apa yang sekarang dapat Anda jelaskan dengan contoh?
2. Apa perbedaan terbesar yang Anda rasakan antara kode sinkron dan asinkron?
3. Error fetch atau JSON apa yang paling membantu pemahaman?
4. Keputusan teknis apa yang Anda ambil dan apa alasannya?

5. Bagian mana yang diselesaikan tanpa bantuan?
6. Umpan balik peer apa yang diterapkan?
7. Kompetensi apa yang menjadi target modul berikutnya?

6. Rubrik Penilaian Dosen

Kriteria	Skor Rendah	Skor Cukup	Skor Tinggi	Bobot
Logika dasar	Tipe dan kondisi sering salah.	Alur utama benar.	Nilai batas dan konversi teruji.	15%
Function dan data	Kode berulang; data tersebar.	Function dan array bekerja.	Tanggung jawab jelas dan reusable.	15%
DOM dan event	Selector atau handler tidak stabil.	Interaksi utama bekerja.	State konsisten dan aksesibel.	20%
Asynchronous	Tidak memahami Promise atau state.	fetch berjalan pada kasus normal.	Error, empty, retry, dan klik ganda teruji.	20%
Form	Validasi tidak lengkap.	Aturan utama bekerja.	Pesan dan state submit aksesibel.	10%
Debugging	Tidak ada bukti uji.	Kasus normal diuji.	Kasus gagal dan bukti diagnosis lengkap.	10%
Proses	Tidak dapat menjelaskan kode.	Menjelaskan dengan bantuan.	Menjelaskan dan mengubah secara mandiri.	10%

Nilai akhir mempertimbangkan hasil, proses, bukti pengujian, dan kemampuan menjelaskan kode. Tampilan menarik tidak dapat menggantikan fungsi yang gagal atau kode yang tidak dipahami.

7. Referensi

1. MDN Learn Web Development. Dynamic scripting with JavaScript. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting
2. MDN Web Docs. JavaScript Guide. <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>
3. MDN Web Docs. Document Object Model. https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
4. MDN Web Docs. Introduction to events. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Events
5. MDN Web Docs. Using promises. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises
6. MDN Web Docs. async function. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
7. MDN Web Docs. Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
8. MDN Learn Web Development. Asynchronous JavaScript. https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Async_JS
9. MDN Learn Web Development. Client-side form validation. https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation
10. The Modern JavaScript Tutorial. Promises, async and await. <https://javascript.info/async>
11. freeCodeCamp. JavaScript Algorithms and Data Structures. <https://www.freecodecamp.org/learn/javascript-algorithms-and-data-structures-v8/>